

DAY 1 • LECTURE 1 • 45 MINUTES

The Wreckage

Linux Basics & the Birth of Containers

The mainframe is gone. Time to learn what we're standing on.

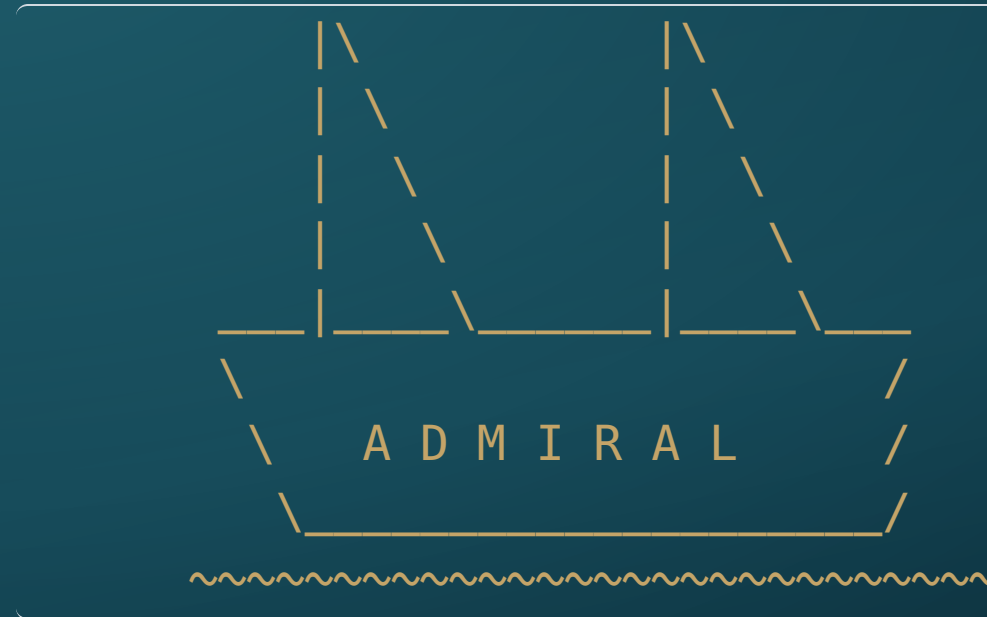


Where we are

- ♦ This morning you built a **ship**: one Ubuntu VM, one `setup-client.sh`, an identical environment for every sailor in the crew.
- ♦ The next 45 minutes: what that ship actually **is** — and the lighter craft we build on top of it.
- ♦ Four short legs:
 - Why the old ship sank
 - The anatomy of a Linux machine
 - How a container actually works
 - How we build one
- ♦ Then we hit the water: **Lab 01 — The First Raft.**

PART I

Why the SS Legacy Sank



"...then all collapsed, and the great shroud of the sea rolled on as it rolled five thousand years ago." — Herman Melville, Moby-Dick

The SS Legacy

"His ship, the SS Legacy, was built around a mainframe which made it slow to change course."

- ♦ One enormous ship. One mainframe. Every function bolted to the same hull.
- ♦ This is a **monolith** — all the code and all its dependencies, shipped and run as a single unit.
- ♦ It floated for years... right up until it had to change.

What a monolith costs you

- ♦ **Change is risky** — edit one feature, rebuild and retest *everything*.
- ♦ **Failure spreads** — one leak floods the whole hull.
- ♦ **Scaling is blunt** — need more of one part? Launch another entire ship.
- ♦ **"Works on my machine"** — the environment is implicit and never written down.

*Faculty translation: thirty laptops, thirty slightly different ships — and an IT queue you don't control.
One bootstrap script routes around both.*

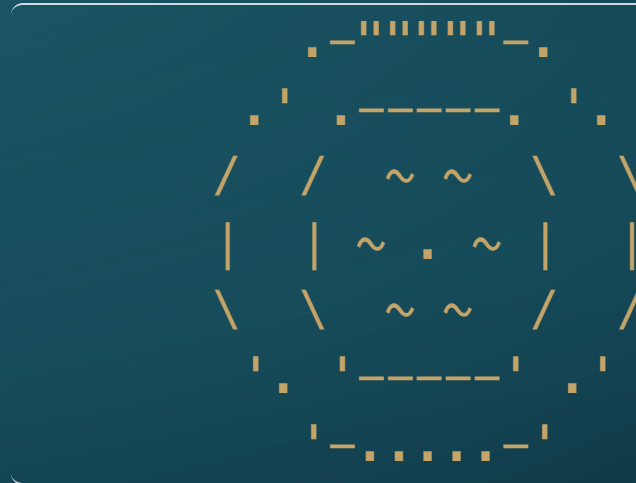
A new approach

"They broke the work down into smaller problems, creating tiny, easily replaceable tools and building supplies."

- ♦ Stop trying to build one unsinkable ship.
- ♦ Build **many small craft** — each does one job, each replaceable without dragging the rest down.
- ♦ Those small craft have a name: **containers**.
- ♦ The rest of today is learning to build and sail them.

PART II

Anatomy of Your Vessel



"Know every plank of your hull, or the sea will teach ye — the hard way." — the Boatswain

Every Linux system has two halves

```
+-----+
|  USER SPACE      the deck – where you work  |
|  fish . starship . docker . kubectl . aichat |
+-----+
|  THE KERNEL      the hull – Linux itself     |
|  processes . memory . files . devices       |
+-----+
|  HARDWARE        CPU . RAM . disk . network  |
+-----+
```

- ♦ The **kernel** runs the hardware. **User space** is every program you launch.
- ♦ Hold this two-layer picture — the whole container idea rides on it.

The Kernel — the hull

- ♦ The privileged core of the operating system. You stand on it; you never touch it directly.
- ♦ Its job:
 - talk to the **hardware**
 - **schedule** which program runs when
 - hand out **memory**
 - own the **filesystem**
 - enforce who is allowed to do what
- ♦ Exactly **one** kernel runs the machine. Everything on board shares it.

Remember that last point — it is the entire reason containers are light.

User Space — the deck

- ♦ Everything above the kernel — every program the crew actually runs.
- ♦ Everything `setup-client.sh` installed this morning lives here: Fish, Starship, Docker, `kubectl`, `aichat`.
- ♦ Programs can't poke the hardware directly. They **ask the kernel**, through **system calls**.
- ♦ An app, plus the libraries and files it needs to run = a **user-space environment**.

Everything is a process

- ♦ A **process** is simply a running program.
- ♦ `setup-client.sh` was a process. The shell you are typing into is one right now.
- ♦ Each process has an **ID**, an **owner**, and its own slice of **memory**.
- ♦ The kernel decides who runs and when — see it yourself with `ps aux` or `top`.

Hold this thought: *a container is just a process. We will come straight back to it.*

Everything is a file

- ♦ In Linux, **almost everything is a file** — documents, directories, even devices and disks.
- ♦ It all hangs off a **single tree**, starting at the root: `/`.
- ♦ Every file carries **permissions** — who may read, write, or execute it.
- ♦ A container gets its **own private view** of this tree. That is the trick we are building toward.

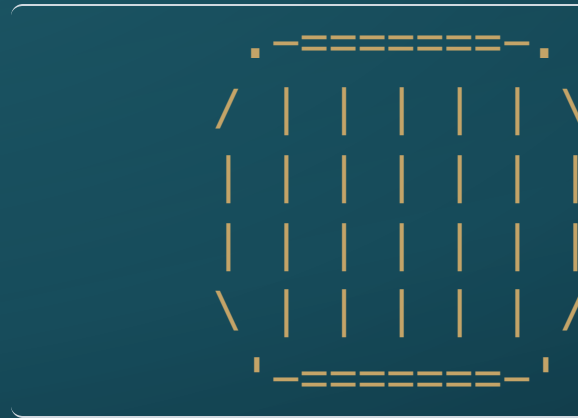
So — what did you build this morning?

```
+-----+
|  USER SPACE   fish, docker, kubectl  |
+-----+
|  ONE LINUX KERNEL                               |
+-----+
|  (virtual) HARDWARE   from VirtualBox |
+-----+
```

- ♦ Virtual hardware → **one** Linux kernel → a user space full of tools.
- ♦ Keep this on screen. Containers do **not** replace any of it.
- ♦ They **reuse the kernel** and add a fresh, isolated user space on top.

PART III

From Hull to Crates

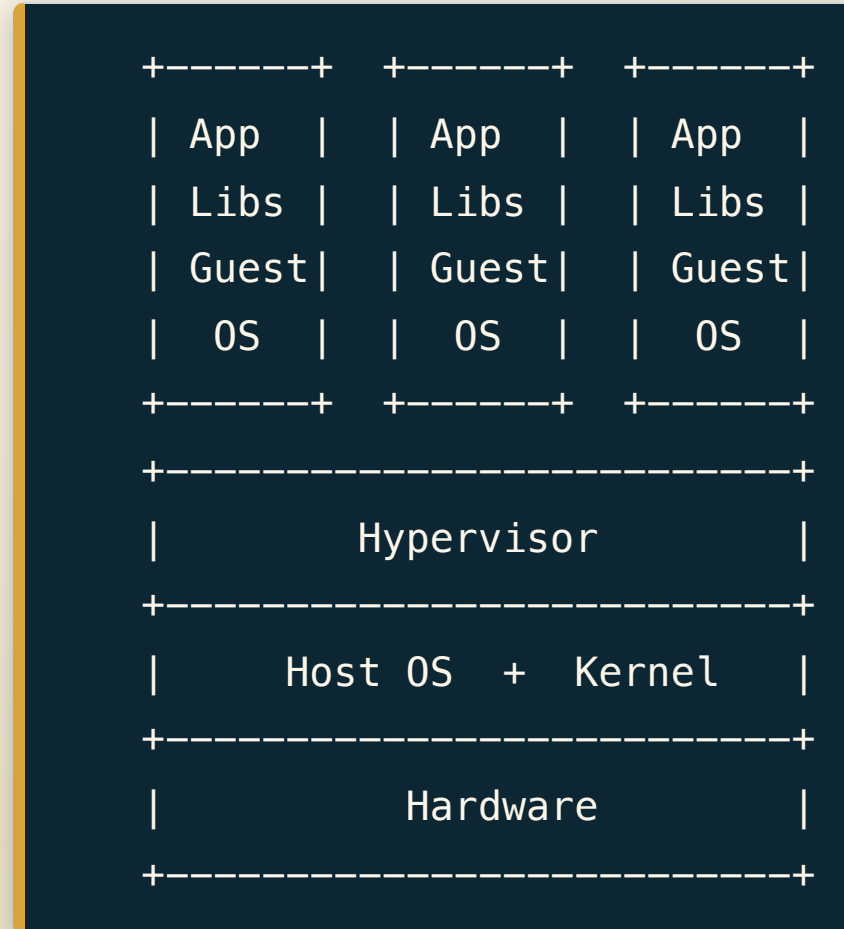


"Why haul a whole ship when a crate'll do? Pack light, sail fast." — the Boatswain

The real problem

- ♦ Shipping your app's **code** is easy. Shipping the **environment it needs** is the hard part.
- ♦ App A needs library version 1. App B needs version 2. Same ship — now they fight.
- ♦ "*Works on my machine*" really means: my environment differs from yours, and **neither is written down**.
- ♦ We need to package the app **together with its whole user space**.

First instinct: give every app its own ship



- ♦ It works — full isolation. But every ship hauls a **whole Guest OS**:
- ♦ **gigabytes of disk · minutes to boot · its own kernel to patch.**

The insight: share the hull

```
+-----+ +-----+ +-----+
| App   | | App   | | App   |
| Libs  | | Libs  | | Libs  |
+-----+ +-----+ +-----+

+-----+
| Container Engine |
+-----+

+-----+
| ONE shared Kernel |
+-----+

+-----+
| Hardware          |
+-----+
```

- ◆ Does each app really need its own **kernel**? **No**.
- ◆ It needs its own **user space** — its own files and libraries.
- ◆ A **container** is an isolated user space running on the **host's** kernel.

Isolation: Linux namespaces

- ♦ A container is just a process — so how is it walled off? With **namespaces**.
- ♦ Namespaces control what a process is allowed to **see**:
 - its own **process list**
 - its own **filesystem** tree
 - its own **network**
 - its own **hostname**
- ♦ Inside the crate, the program is convinced it is alone on the ship.

Limits: Linux cgroups

- ♦ Namespaces decide what a process can **see**. **Control groups** (cgroups) decide what it can **use**.
- ♦ A ceiling on **CPU time**, **memory**, and **I/O** — enforced by the kernel.
- ♦ One greedy crate can't hog the whole deck and capsize its neighbours.
- ♦ Namespaces + cgroups are both **ordinary kernel features**. Nothing exotic — Linux already had them.

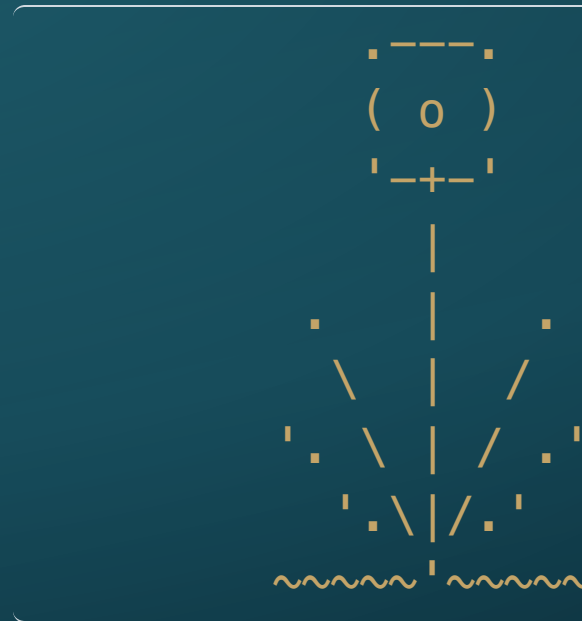
So what is a container?

- ♦ A container is **not a tiny virtual machine**.
- ♦ It is: a normal **Linux process** + **namespaces** (what it sees) + **cgroups** (what it uses) + its own packaged **user space** — all riding the **shared host kernel**.
- ♦ No Guest OS. No boot sequence. It starts in **milliseconds**.

The Boatswain: *"A container's just a wooden crate, lad. The hull is the OS, the loading docks are the ports — the crate only carries the cargo."*

PART IV

Building a Crate



"Measure twice, cut once — a crate built right is a crate built once." — the Boatswain

Image vs. Container

- ♦ **Image** — the blueprint *and* the parts: a frozen, **read-only** package of a user space.
- ♦ **Container** — a **running instance** of an image.
- ♦ One image → **many** identical containers. Build once, run anywhere, the same every time.

The blueprint stays in the drawer. The crates go in the water.

Images are built in layers

```
harbor.wagbiz.org/raft-fleet/blackbeard:v1
```

+-----+	
COPY index.html	your flag (layer 3)
+-----+	
install nginx	the web server (layer 2)
+-----+	
FROM alpine	a 5 MB base hull (layer 1)
+-----+	

- ◆ Each instruction in the recipe adds **one layer**, stacked bottom-up.
- ◆ Layers are **cached and shared** — change only your `index.html`, and only the top layer rebuilds.

The Dockerfile — the blueprint

- ♦ The blueprint is a plain-text file: the **Dockerfile**.
- ♦ A short recipe — for example, **FROM** a base image, then **COPY** your files in.
- ♦ Written once; it produces the **exact same image** on any machine, anywhere.

*We won't drill the syntax on a slide. In **Lab 01**, the **Socratic Boatswain** — your AI crewmate — coaches you through writing it yourself.*

The Harbor — where images live

```
Dockerfile  --build-->  Image  --push-->  Harbor
(blueprint)          (crate)          (the docks)
                    |
                    +-----run-----> Container
                                      (raft afloat)
```

- ♦ A finished image needs a home: a **registry** stores and shares images.
- ♦ Your island's registry is **Harbor**, at `harbor.wagbiz.org`.
- ♦ Build it, push it to the docks, and the cluster can pull it whenever it needs to.

Instructor Superpower

THE REPRODUCIBLE ENVIRONMENT

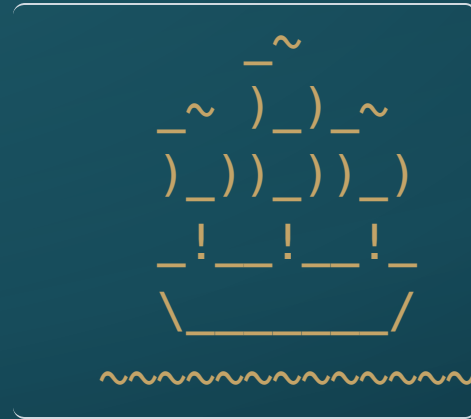
- ♦ This morning's `setup-client.sh` and this afternoon's container are the **same idea at two scales**:
- ♦ *describe the environment as code → build it once → hand everyone an identical copy.*
- ♦ Containerize an assignment and you ship students the **environment**, not just the code — thirty identical setups from one definition.
- ♦ The environments you can choose to support have radically expanded. You don't wait for IT to bless a toolchain — you build it into the image yourself.

Up next: Lab 01 — The First Raft

- ♦ Write a **Dockerfile** for an Nginx web server.
- ♦ Hoist your flag — a custom **index.html** with your name on it.
- ♦ **Build** the image, **test** that your raft floats, **push** it to Harbor.
- ♦ Meet your AI crewmate: the **Socratic Boatswain** (it will *not* hand you the code).

Success looks like one thing: your raft afloat and docked in the Harbor — ready for the cluster to find it this afternoon.

To the rafts.



The hull is yours. Now build something that floats.

